

Cryptography - 7_10

Pseudorandomness

Our first step towards efficient symmetric crypto. Moreover, pseudorandomness is used in modern computers to "simulate" real randomness.

We will see that OWF (one way functions) are enough for pseudorandomness.

✓ Definition

A function $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$ is one-way, if

$\forall PPT \mathcal{A}$:

$$\Pr_{x \in \{0,1\}^n} [f(x') = y : y = f(x); x' \leftarrow \mathcal{A}(y)] \leq \text{negl}(n)$$

(where $\mathcal{A}$ is the adversary)

($x' \leftarrow \mathcal{A}(y)$: x' is outputted by the Adversary with y)

So the probability has to be negligible, which means that it goes to zero faster than any inverse of a polynomial function.

≡ Example of $\text{negl}(n)$

$$2^{-n}$$

🔗 Exercise

Prove $2^{-n} = \text{negl}(n)$

🔗 Exercise 2

$\mathcal{E}_1(n), \mathcal{E}_2(n) = \text{negl}(n)$, then $\mathcal{E}(n) = \mathcal{E}_1(n) + \mathcal{E}_2(n)$ also $\text{negl}(n)$.

🔗 Exercise 3

$p(n) \cdot \mathcal{E}(n) = \text{negl}(n)$ if $p(n) = \text{poly}(n)$ and $\mathcal{E}(n) = \text{negl}(n)$.

Alternative: We can think of this, giving an adversary $\mathcal{A}$ "playing" with a challenge (trying to win, breaking the OWF).

The Challenger $\mathcal{C}$ picks $x \in \{0, 1\}^n$ and gives y to $\mathcal{A}$, $\mathcal{A}$ picks an x' and wins only if $y = f(x')$.

Let's now define pseudorandomness.

This means sequence of bits that are not random but they look random. We capture this requirement using indistinguishability (computational).

We have already seen something like this, the statistical distance. Given X, X' random variables over the same domain, then $SD(X; X') \leq \epsilon$ is equivalent to say:

$$\forall \mathcal{D} : |Pr[\mathcal{D}(x) = 1 : x \in X] - Pr[\mathcal{D}(x) = 1 : x \in X']| \leq \epsilon$$

($\mathcal{D}$ is the distinguisher (?), we're computing the probability that the Adversary can tell two distributions apart)

Alternative: Same as before, the challenger $\mathcal{C}$ gives an x to the distinguisher $\mathcal{D}$, and he sends a bit $b' \in \{0, 1\}$, either one or zero, to distinguish if x is in X or X' .

✓ Definition

$\stackrel{\equiv X}{\{X_n\}}, \stackrel{\equiv Y}{\{Y_n\}}$ (with those being a family of distributions over the security parameter n) are computationally indistinguishable ($X \approx_c Y$, X is approximately Y):

$$\forall PPT \mathcal{D} \ Pr[\mathcal{D}(z) = 1 : z \in X] - Pr[\mathcal{D}(z) = 1 : z \in Y] \leq \text{negl}(n)$$

🕒 Exercise 4

If $X \approx_c Y$ and $Y \approx_c Z$ then $X \approx_c Z$.

With this, we can define pseudorandom generators.

It's something that takes as input a short string (with length being n , the security parameter, like 128 bits), and it outputs a sequence even longer (even terabytes). Yet, this is indistinguishable from random.

✓ Definition (PRG)

A PRG $G : \{0, 1\}^n \rightarrow \{0, 1\}^{n+l}$ with $l \geq 1$ (the stretch) is secure if:

$$G(U_n) \approx_c U_{n+l}$$

(U_n is the uniform over $\{0, 1\}^n$, similar for U_{n+l})

Alternative: We have that $\mathcal{D}$ receives a z , which is either $G(s)$ or u , with $s \in \{0, 1\}^n$ and $u \in \{0, 1\}^{n+l}$. Then $\mathcal{D}$ outputs $b' \in \{0, 1\}$, and the probability that he guess it is at most $\text{negl}(n)$.

Looking ahead: A PRG is useful to do symmetric key encryption beating Shannon. Now $\text{Enc}(k, m) = G(k) \oplus m = c$, and $\text{Dec}(k, c) = G(k) \oplus c = m$.

We now have, in fact, that $|k| \ll |m|$ (we were stuck before with keys with the same length of a message, now we can use short keys).

Before doing that, let's understand how to build PRGs.:

1. Use a randomness extractor to get a uniform seed $s \in \{0, 1\}^n$.
2. Define a simple PRG $G : \{0, 1\}^n \rightarrow \{0, 1\}^{n+1}$ with minimal stretch (just one bit, $l = 1$).
3. Use G to stretch any $l(n) = \text{poly}(n)$.

Theory vs Practice:

- Randomness extraction is what we already studied. But in practice it is done using hash functions.
- Theoretical G can be obtained from any OWF. Practical G is heuristic.
- Stretch amplification is the same.
- (In practice the seed is refresh periodically collecting new entropy) - we will not cover that.

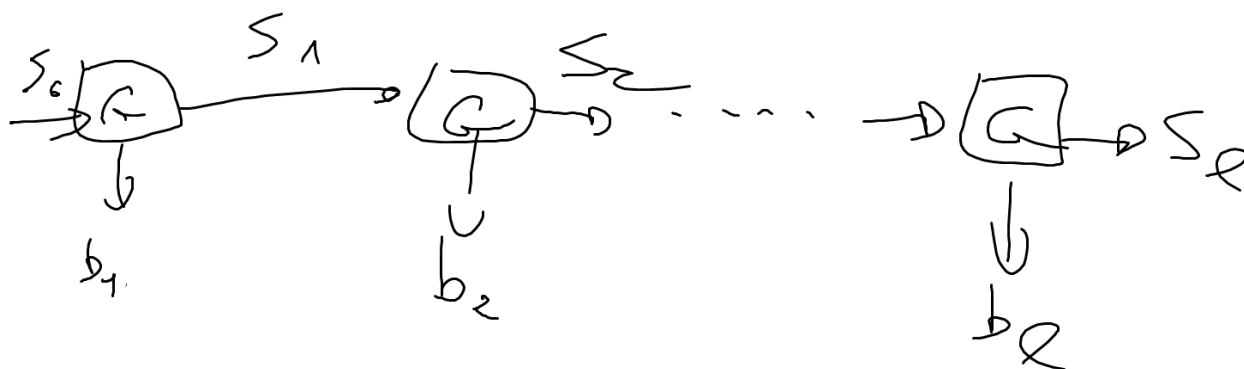
✓ Theorem

If there exists a PRG $G : \{0, 1\}^n \rightarrow \{0, 1\}^{n+1}$, then there exists a PRG $G^l : \{0, 1\}^n \rightarrow \{0, 1\}^{n+l}$ for any $l(n) = \text{poly}(n)$.

Alternative:

We iterate G , running with an initial seed s_0 , getting $n + 1$ bits, with the last we call $b_1 \in \{0, 1\}$, and the rest is s_1 , that we put inside G again, outputting b_2 and so on, until I output s_l and so b_l .

So $G^l(s_0) = (b_1, \dots, b_l, s_l)$.



Proof

Assume G_l not secure, then $\exists$ PPT $\mathcal{D}'$ that can distinguish $G^l(U_n)$ from U_{n+l} with probability $\geq \frac{1}{p(n)}$ for some polynomial $p(\cdot)$ (so not negligible).

We want to build PPT $\mathcal{D}$ that can distinguish $G(U_n)$ from U_{n+1} with probability $\frac{1}{p(n)}$.

$\mathcal{D}$ would be a reduction (in complexity theory).

We have then $\mathcal{D}'$ and $\mathcal{D}$, with the last one talking to the challenger $\mathcal{C}^{prg}$. He is given a

z , which could be either $G(s)$ (with $s \in \{0, 1\}^n$) or U_{n+1} . $\mathcal{D}$ then speaks a certain language, receiving z of length $n + 1$, $\mathcal{D}'$ speaks another language, expecting to receive a string of length $n + l$. How do I do this (where do I put z)?

We use a technique known as hybrid argument.

We have that $H_0(n) \equiv G^l(U_n)$, and at the other extreme we have $H_l(n) = U_{n+l}$. In the middle, we start from a point i , where we sample uniformly till i . So we have that

$$H_i(n) \equiv \begin{matrix} 1, \dots, b_i \in \{0, 1\} \\ s_i \in \{0, 1\}^n \\ (b_{i+1}, \dots, b_l, s_l) = G^{l-i}(s_i) \end{matrix}$$

So I reduce the problem of proving H_0 indistinguishable to H_l , that for every i H_i is indistinguishable from H_{i+1} . We use the photo above starting from a different point in the "chain" of G .

Lemma

$$\forall i : H_i \approx_c H_{i+1}.$$

Proof

By reduction (as before).

I assume $\mathcal{D}'$, $\mathcal{D}$ and $\mathcal{C}$, as well as the other things. In both H_i and H_{i+1} the first i bits are uniform, so $b_1, \dots, b_i \in \{0, 1\}$ are uniform (true in both H_i and H_{i+1}).

Now I must decide b_{i+1} . Because I have z , I take it and parse it as $s_{i+1} || b_{i+1}$. If b_{i+1} comes from the PRG, then it's like H_i . If it's uniform, then it comes from H_{i+1} .

How do I define now $(b_{i+2}, \dots, b_l, s_l)$? I use $G^{l-i-1}(s_{i+1})$, we are starting from $i+1$.

This is true in both H_i and H_{i+1} .

After I'm done speaking with $\mathcal{D}'$, he will be able to distinguish, and gives an output as a bit.

By the above observations,

$$Pr[\mathcal{D}(z) = 1 : z = G(s); s \in \{0, 1\}^n] = Pr[\mathcal{D}'(b_1, \dots, b_l, s_l) = 1 : (b_1, \dots, b_l, s_l) \in H_i(n)]$$

z comes from the PRG, s is uniform

when z comes from the PRG

$$Pr[\mathcal{D}(z) = 1 : z \in U_{n+1}] = Pr[\mathcal{D}'(b_1, \dots, b_l, s_l) = 1 : (b_1, \dots, b_l, s_l) \in H_{i+1}(n)]$$

This implies that

$\implies$

$$|Pr[\mathcal{D}(z) = 1 : z = G(U_n)] - Pr[\mathcal{D}(z) = 1 : z \in U_{n+1}]| \geq \frac{1}{p'(n)}$$

which is non negligible. We have proven that $H_i \approx H_{i+1}$, thus reaching a contradiction in the above proof.

■